GIL - Get in Line

Week 9 | From 4/7/26 – 4/14/26

Leader: Neel Vavilapalli

Disha Shetty ~17 hours **Time Spent**	**Tasks Completed** **1. System Testing & Bug Fixes (~5 hours)** • Identified and debugged critical queue display error affecting waitlist visibility • Reverted to stable deployment version to maintain system reliability • Fixed invalid date handling on profile page • Resolved text wrapping issue in reverted version • Conducted comprehensive testing to validate all fixes **2. Scalability Documentation (~3 hours)** • Documented three-pillar architecture (React frontend, Node.js API, FastAPI AI service) • Detailed local-first design, sync strategy, and conflict resolution • Explained rate limiting implementation (120 requests/minute per IP) • Documented CI/CD pipeline for Vercel and Render deployments • Analyzed load balancing and horizontal scaling capabilities • Wrote conclusion with production scaling recommendations **3. Infrastructure Research (~1 hour)** • Investigated Vercel and Render scaling limitations • Identified Render's free tier constraint: single-instance with 20 concurrent tasks • Confirmed Vercel's automatic edge network scaling for frontend • Identified backend as primary scalability bottleneck **4. Team Coordination (~2 hours)** • Participated in team meeting for project alignment **5. Additional Bug Fixes (~6 hours)** • Fixed broken archive functionality • Corrected seating logic — waitlist prop in StaffDashboard was holding entries for all events combined instead of per-event

	• Resolved critical ID reconciliation issue — refresh was only updating the user side without syncing properly • Fixed removeFromWaitlist never calling Supabase • Fixed addToWaitlist using fire-and-forget with a local ID that was never reconciled with the server • Fixed loadEventsFromSupabase wiping offline-created events on load • Fixed syncEventToSupabase never storing the returned Supabase UUID • Replaced sync.ts stub endpoint with working implementation • Fixed occupiedTables and guestsSeated being hardcoded to 0 • Replaced clear-table stub with working implementation • Added navigator.onLine checks throughout the frontend (previously absent) • Fixed loadEventsFromSupabase being called without await • Addressed stateless refresh tokens that made revocation impossible • Fixed no_show_policy sending a number instead of the required string type **Challenges Faced** 1. **Queue Display Bug:** Complex issue required extensive debugging and eventual rollback to maintain availability 2. **Scalability Constraints:** Render's single-instance limitation creates a fundamental bottleneck without an infrastructure upgrade 3. **Time Management:** Bug fixes took longer than expected, impacting the documentation timeline 4. **Sync & Data Integrity Issues:** Multiple layered bugs in the sync pipeline required careful untangling to avoid data loss across offline and online states **Planned Tasks** • Implement max pooling to limit users and prevent app crash • Implement "SERVER_WINS" resolution strategy, replace current boilerplate function. • Identified 3-4 more bugs that need fixing

Angela Lopez
 	
Time Spent ~ 15 hours
	Tasks completed:
	• Weekly Meeting
	• Ran tests from user side
	  o Joining capacity-based waitlists
	  o Joining table-based waitlists

- Ran tests from staff side
    - Creating/editing/deleting events
        - Table-based
        - Capacity-based
        - Attendance-based
- Added filter to reservation list
- Fixed the syncing error when selecting the # of seats for each table
- Added time and date to reservation cards
    - Also added to the events in “My Events”
- Made edits to “Profile” page:
    - Deleted “Member since...”
    - Added phone number formatting along with formatting hint
- Added back button to all event pages
- Merged updated Figma version with current GitHub version
- Added new column to “party” in Supabase: “reservation_time”
- Fixed user view problem -> now takes user to “Welcome” screen after logging in, instead of one of their events
- Reservations or people on the waitlist correctly syncing with backend, now disappear from list after being seated (before, they would reappear after a few seconds)
- Occupied tables on “Your Events” page syncing properly with # of tables seated on “Table Management” page
- Fixed event name to properly show entire name(was only showing first letter and ellipses)
    - Also limits character count to 20 now
- Added in public/private toggles for all events
- Added in “special request” logic -> should take user requests into consideration when seating

Planned tasks for next week:

- Make sure that public/private toggles being represented correctly in user view
- Make sure events are going to “Past Events” after user is seated
- Continue testing and modifying single and multiple queue logic
    - Not correctly syncing, will revert to empty queues after some time
- Make sure “seated” notification being shown on user side
- Event (staff side) and reservation (user side) edits not being saved correctly
- Make sure table seating logic is correct
    - If a table is added to the event, it should keep the seated guests or give a notification that a table cannot be removed because they are already occupied

Any issues or challenges:

- AI credits on Figma Make are limited so the prompts given have to be very specific in order to not “waste” them

	• Integrating frontend and backend in GitHub and making sure it was all syncing correctly o There were a lot of parts in the current GitHub version of the frontend code that were hardcoded, so even though the updated Figma code was correct, it was being overridden after a few seconds
Neel Vavilapalli Time Spent : 11 hours	Tasks completed: **Weekly Meeting:** Participated in the weekly team meeting to align on project milestones, coordinate backend integration with front-end requirements, and review the current development roadmap. **Service Layer Refactoring:** Migrated all data persistence and retrieval logic in `services.py` to use the Supabase Python client. This replaces local/mock data handling with real-time database operations, ensuring thread-safe connections and structured query building. **Entry-Point Optimization:** Updated `main.py` to initialize the Supabase client and pass dependencies through to the FastAPI routes. This ensures that the AI server maintains a persistent connection to the data layer without redundant overhead. **WaitTime.ts Modernization:** Deprecated the client-side linear approximation logic. The frontend now functions as a consumer of the backend's heuristic model, which accounts for historical "no-show" rates and rolling 12-month reliability scores. **Model Syncing:** Refactored various frontend hooks to listen for backend updates. This addresses previous state-drifting issues where the UI would display stale time estimates while the server was recalculating. **API Standardization:** Updated TypeScript interfaces and service functions to match the new JSON response shapes coming from the FastAPI server. Planned tasks for next week: • Push changes to github so they show up in deployment Any issues or challenges: • Sync issues

Kyle Albuquerque
 	
Time Spent 11 hours
	Tasks completed:
	• Weekly Meeting
	• Worked on and reviewed the Scalability document
	• Researched ways to Stress Test our application
	  o K6
	  ♣ TA recommendation
	  o Apache Jmeter
	  ♣ Might be able to also help with load balancing
	  o Locust
	  ♣ Possibly too complicated for our project specifications
	  o Gatling
	• Frontend fixes (not pushed on repo)
	  o Added some Verification/Checks so that it would send good data to the backend
	  ♣ Checks also occur in backend but this is just to be safe
	  o Looking into some functions that are causing sync errors with backend
	  ♣ Frontend and backend are both trying to do the same thing in some areas, but in different ways
	• Working on frontend/backend connection (not pushed on repo)
	  o Connecting functions where they need to be called
	  o Editing parameters that the frontend is sending to backend
	• Performed general bug hunting through exploratory testing
	• Tested different user flows to identify edge cases and potential failure points
	• Refactored small sections of code for better readability and maintainability
	
	Planned tasks for next week:
	• Continue to look through the frontend and implement the correct function calls where needed
	• Will go through and improve error handling between frontend and backend communication
	• Use desired platform to conduct stress testing of the app based on research findings
	
	Any issues or challenges:
	• Getting the app to run locally was giving problems due to the

	way windows/mac process strings - Was unable to locate my .env so it was not loading up correctly and would error out - There is a lot of issues with the way frontend and backend are currently connected so I am trying to fix all of that locally before pushing so that it does not cause any troubles with the auto-deployment

Total Time Spent: ~54 hours

Summary:

This week, the team made significant strides in stabilizing the core infrastructure and resolving critical synchronization issues within the event orchestration platform. A major focus was placed on **reconciling the frontend and backend**, with extensive work performed on the Supabase integration to ensure that offline-created events and waitlist entries are properly synced with the server. Key bug fixes addressed broken archive functionalities, seating logic errors where waitlists were incorrectly combined across events, and several "fire-and-forget" ID reconciliation issues that previously led to data loss. On the backend, the service layer was refactored to use the Supabase Python client, and the entry-point was optimized to ensure thread-safe, persistent database connections for the FastAPI server.

In addition to technical fixes, the team prioritized **scalability and performance research**. This involved documenting a three-pillar architecture (React, Node.js, and FastAPI) and analyzing the limitations of deployment platforms like Vercel and Render. Detailed scalability documentation was produced, covering local-first design strategies, rate limiting, and CI/CD pipelines. The team also conducted exploratory stress-testing research, evaluating tools like K6 and JMeter to prepare for future load balancing. On the UI/UX front, the "Profile" and "My Events" pages were updated for better clarity, phone number formatting was standardized, and a new heuristic model was integrated to replace client-side approximations with more accurate, server-side wait-time estimates.